Young-Hyun Kwon

PROJECT OVERVIEWS

Project Name: Synthesis and Image Editing with Diffusion
Project Duration: 2025.01.06 – 2025.03.14
Tech Stack: Python, PyTorch, HuggingFace Diffusers, NumPy, Matplotlib

This project aims to directly implement and apply the probabilistic image generation process of Diffusion models.
Using DeepFloyd IF as the base model, I experimented with **text-to-image generation**, **inpainting**, and **image restoration**, reproducing the **forward and reverse sampling** processes both mathematically and in code.

By adjusting major hyperparameters—sampling steps, noise scheduling, and Classifier-Free Guidance (CFG) strength—I analyzed the trade-offs between image quality and diversity.
To overcome GPU memory limitations, I applied **half-precision operations and inference step optimization**, constructing an efficient generation pipeline.

TECHNICAL CHALLENGES

Main Goal:
Implement the stochastic denoising process of diffusion models at the code level while maintaining stable generation quality within limited compute resources.

1. Mathematical Implementation of Sampling Loop
   - Implemented both the forward (noise addition) and reverse (denoising) processes without relying solely on library functions
   - Experimentally validated β-scheduling, variance computation, and timestep-dependent noise prediction
   - Visualized intermediate results to analyze stability and convergence

2. Classifier-Free Guidance Optimization
   - Studied how changes in CFG affect image fidelity and diversity
   - Conducted experiments with CFG values from 1 to 12
   - Found CFG = 7 produced the most balanced results

3. GPU Memory Limitations & Optimization
   - Frequent out-of-memory interruptions in Google Colab due to VRAM ≈ 12GB
   - Applied FP16, gradient checkpointing, and reduced inference steps
     Achieved ~40% memory reduction and >30% faster execution

4. Inpainting & Restoration Extensions
   - Added mask-based region restoration to the pipeline
   - Resampled noise around boundaries to maintain texture consistency

CONTRIBUTIONS

- DImplemented diffusion sampling, iterative denoising, and CFG logic from scratch
- Built DeepFloyd IF Stage I & II pipelines using PyTorch and HuggingFace Diffusers
- Visualized forward/reverse processes for quantitative noise analysis
- Extended functionality to support **Inpainting** and **Image-to-Image Translation**

OUTCOME

Text-to-Image Generation

- Prompt-based image synthesis with adjustable CFG (3–10) and steps (10–50)
- Best performance: CFG = 7, steps = 30
- Example prompts:

- “An oil painting of a snowy mountain village”
- “A man wearing a hat”
- “A rocket ship”

Inpainting

- Natural restoration of masked/damaged areas
- Boundary noise resampling preserved lighting and background texture consistency

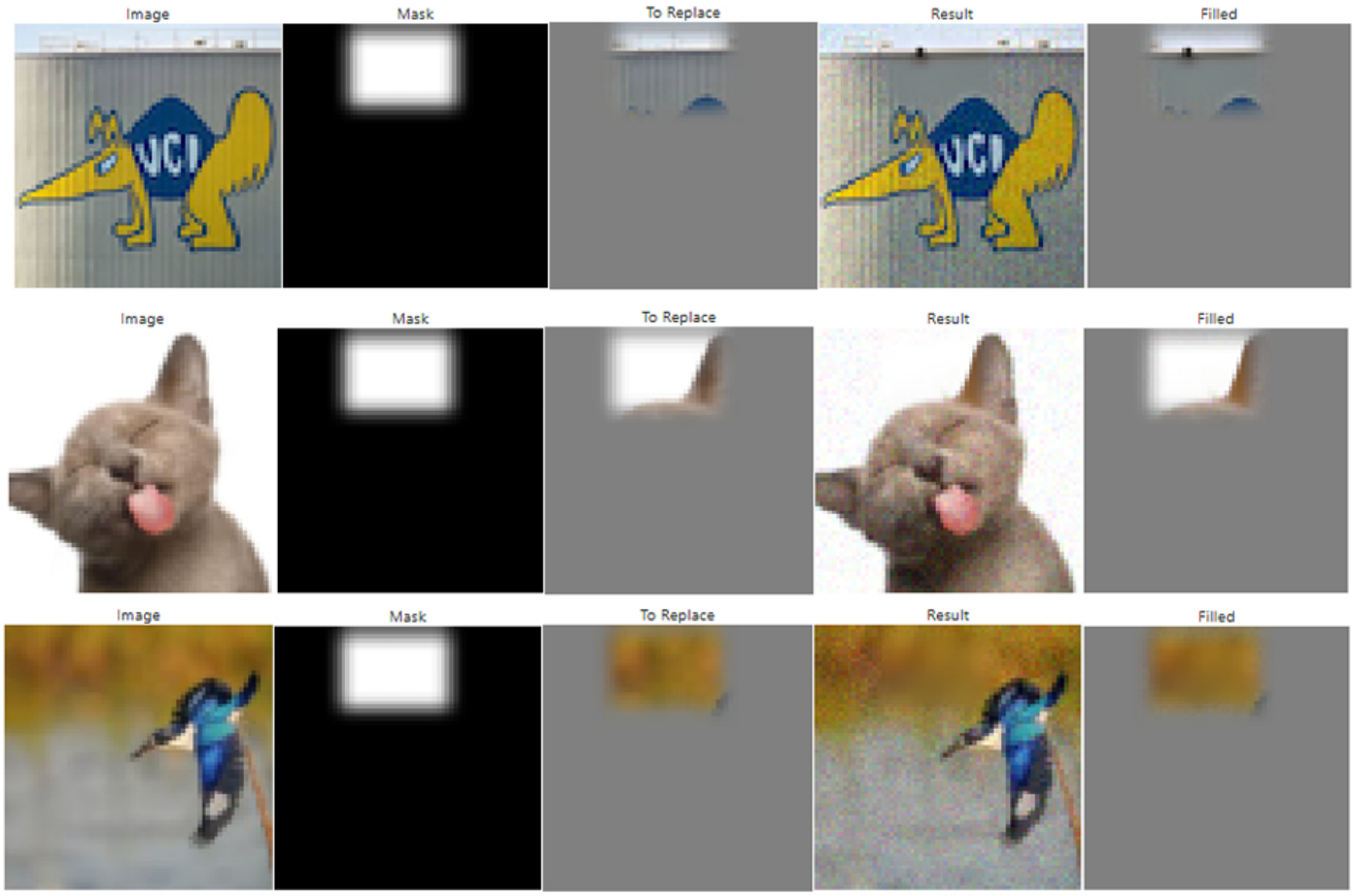
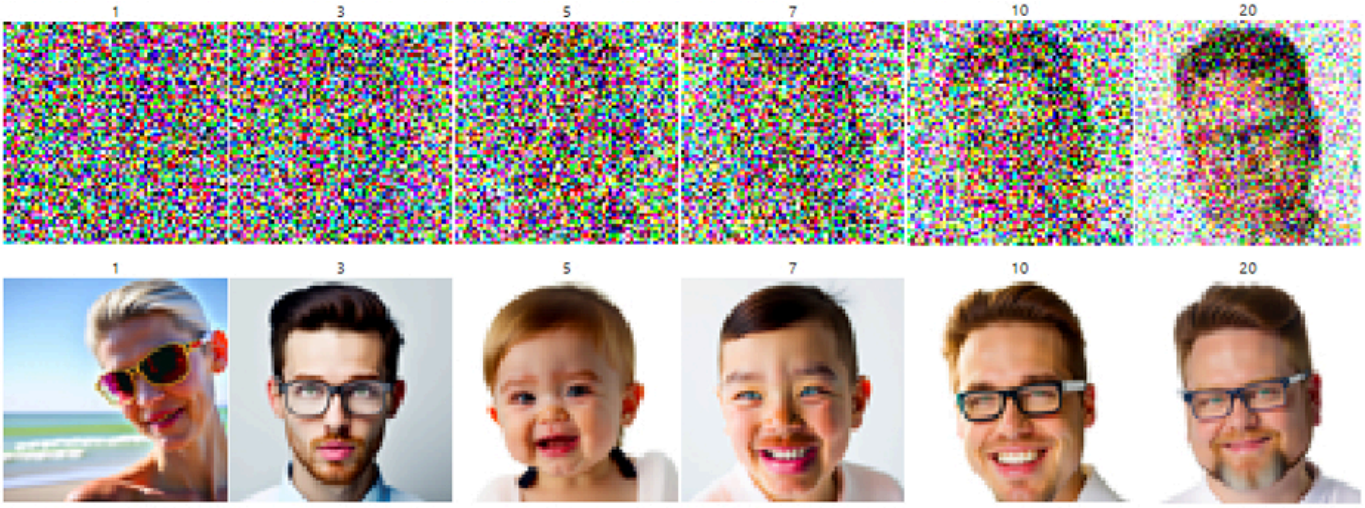


Image Restoration / Denoising

- Added progressive noise then restored images via reverse diffusion
- Higher step counts improved detail and edge clarity
- Demonstrated strong effectiveness for Gaussian noise removal

Image-to-Image Translation

- Style and composition adjustments using text prompts
- Balanced transformation strength by tuning sampling steps
- Verified utility of diffusion in unsupervised style transfer
- Prompt: "a high quality photo"



LIMITATIONS & FUTURE IMPROVEMENTS

1. GPU Memory Constraints
   - Stage II execution and high-resolution generation remain limited by VRAM
     → Plan: lighter execution architecture with auto-scaling resolution

2. Non-linear Behavior of CFG
   - Excessive CFG (>10) led to color distortion and unstable visuals
     → Plan: adaptive CFG scheduling to maintain stability

3. Prompt Interpretation Limitations
   - Some detailed prompts underperformed due to text encoder (T5) constraints
     → Plan: keyword emphasis + improved semantic understanding

4. Boundary Consistency in Inpainting
   - Occasional unnatural color blending around mask edges
     → Plan: post-processing blending algorithms for smoother transitions

TAKEAWAYS

- Gained deep mathematical understanding of diffusion-based probabilistic generation
- Learned balanced tuning of control variables (CFG, sampling steps)
- Improved engineering skills through optimization, error handling, and data post-processing
- Acquired quantitative evaluation techniques for understanding hyperparameter impact
- Strengthened practical expertise in implementing state-of-the-art generative models

Detailed experiment logs and analysis are available in a separate report upon request